This programming project is due on Wednesday, December 10, 2025

Reminder: Do your own work on this project. Do not obtain any code from another student, from an AI-based code generator or from the Internet in general. Do not show your code to anyone except the instructor, or an official BHCC Tutor. Refer also to the last page of the course *Syllabus*, for details about the BHCC policy regarding academic dishonesty.

Be sure that you read and understand this entire document before you begin writing your code. Pay close attention to the **Project Deliverables** and **Grading Criteria** sections of this document. If you have **questions**, ask the instructor during class or contact the instructor by BHCC e-mail: pmorgan@bhcc.edu.

Table of Contents

Simple Drawing Application 1
Project Deliverables: 2
Program Header Documentation 3
General Grading Criteria 3
Neatness Counts 3
Names Matter 3
Structure Matters 3
Detailed Grading Criteria 4

Simple Drawing Application

This project involves writing a JavaFX application that draws short line segments in response to the user typing the arrow keys on the keyboard.

The first line starts from the center of the pane and draws in the direction corresponding to the arrow key that the user types: **RIGHT**, **UP**, **LEFT**, or **DOWN**. The *ending* X, Y position of the line segment becomes the *beginning* X, Y position of the *next* line segment. Each line segment should be 10 pixels long.

If the user holds down the **SHIFT** key while hitting an arrow key, then the code does a MOVE to the new X, Y position, without actually drawing anything. Then, if the user types another (unshifted) arrow key, the next line segment will be drawn starting at the new location.

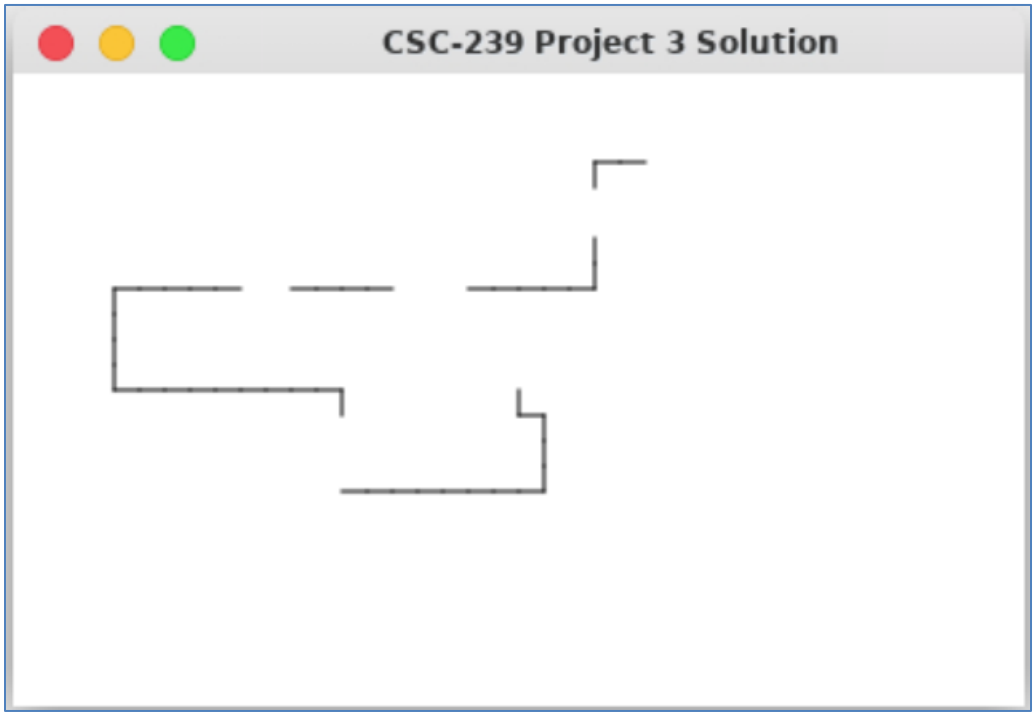
If the user types the **DELETE** key, then the program should erase all line segments.

The KeyCode constants are defined in the

javafx.scene.input.KeyCode

class.

A sample of the display is shown below. if the user holds down the **SHIFT** key while hitting an arrow key, then the code does a MOVE to the new X, Y position, without actually drawing anything. Then, if the user types another (unshifted) arrow key, the next line segment will be drawn starting at the new location.



Project Deliverables:

The project source file(s) should be submitted by Moodle, using the Moodle Activity:

CIT239_Project3

Submit your **`.java`** file(s) that you have created. I will need to compile your code on my home computer in order to grade it. If you are submitting more than one file (**`.java`**), do **NOT** enclose the files in a ZIP file. Moodle will allow you to submit multiple files.

Program Header Documentation

Each source file (**.java** file) that you submit as part of your project should have an appropriate comment at the top of the file. The header should include **at least** the information stated in the example below:

```
/*
*  CSC-239 Project 3:   Simple Drawing Application
*  Student:     yourName
*  Date:        todaysDate
*  Description: This program ...

*/
```

General Grading Criteria

The project will be graded according to the following grading criteria:

Neatness Counts

The program source code should be clearly commented, with relevant information clearly stated, and extraneous garbage removed. "Extraneous garbage" includes:

- **"DEBUG" statements:** while printing intermediate results is an effective debugging technique, these statements should be removed or disabled before the final version is submitted.
- **Failed attempts:** If you have sections of code which you have abandoned because you replaced them with a different approach, then these non-working sections should be removed before submitting the project for grading. It is a good idea to save a copy of the non-working code for yourself, under some other filename, but the code you submit should be "clean". (Naming suggestion: a good name for the "first draft, non-working" version of a project might be: **`project3_first_draft.txt`**. The reason to change the filename suffix from "**`.java`**" to "**`.txt`**" is to prevent your IDE from trying to compile the old version.)

Names Matter

The names chosen for classes, variables, and methods should be descriptive, relevant, not too short and not too long. Do NOT just choose whatever arbitrary name pops into your head. Think about what a variable means, and what role it plays in the program; then, choose a name that helps the reader understand the code.

Structure Matters

The code should be divided into logical methods, without too much code in the "main" method, or any other method.

Detailed Grading Criteria

The project will be graded according to the following grading criteria:

Feature	Portion of grade
1. The program functions correctly.	65%
2. The program is clearly organized and **commented** so that it is easy to read and understand. At a minimum, there should be a comment at the beginning of each method that explains what that method does. Use your judgement regarding any additional comments that may be needed to make the program easy to understand, without over-commenting the program. (As you get more experience, your judgement about this will improve.) Do **NOT** put all of your code in the **main** method or any other method.	10%
3. Use good variable names and method names: • A variable name or method name should indicate something about what that variable or method does in the program. • Variable names and method names should be not too short and not too long.	10%
4. Cleanup any unused portions of code, such as "failed attempts" that you later replaced.	5%
5. Place a brief summary of the program in comments at the beginning of each source file. Also be sure these comments have your name and the due-date for the project.	5%
6. Cleanup any irrelevant comments	5%
Total:	100%

Copyright © 2025 Peter Morgan. All rights reserved. You may **not** share this document with anyone or use it in any way other than as a participant in this course.